Organización del repositorio y resolución de conflictos

Trabajo Final Integrador — Sistema de reservas del SUM Municipal de Bella Vista

Informe técnico y memoria del trabajo colaborativo

Fecha de revisión: 13 de octubre de 2026.

Participantes identificadas en el historial: Emilia Gomez y EsmeraldaMarquez.

Versión analizada: `985ae98ddb53a6b12457f5228ad3ca6987d0eace`.

Alcance: organización del código, decisiones de diseño, uso de ramas, reconstrucción del conflicto y evaluación del resultado integrado.

Índice

1. Propósito y fuentes del informe.
2. Contexto funcional y alcance del sistema.
3. Estructura del repositorio y criterios de organización.
4. Responsabilidades de las capas y recorrido de una reserva.
5. Decisiones de tecnología, dependencias y mantenimiento.
6. Organización de los estilos y conservación del diseño.
7. Trabajo colaborativo y cronología de las ramas.
8. Descripción del conflicto: relato y evidencia técnica.
9. Reconstrucción de los bloques conflictivos.
10. Resolución adoptada y estado final.
11. Procedimiento de resolución y verificación.
12. Aprendizajes y acuerdos de trabajo recomendados.
13. Evaluación de la organización y mejoras posibles.
14. Texto de síntesis para la presentación del trabajo.
15. Anexos: comandos, glosario y fuentes.

1. Propósito y fuentes del informe

Este documento explica cómo está organizado el repositorio del sistema del SUM Municipal, qué problemas resuelve esa organización y cómo se integraron dos líneas de trabajo que modificaron zonas comunes de la interfaz. Su finalidad es dejar una memoria técnica que pueda incorporarse a la documentación del Trabajo Final Integrador y utilizarse para explicar las decisiones del equipo durante una presentación.

El análisis se apoya en el código de la copia local, sus archivos de documentación y el historial de Git disponible. También incorpora el relato de Emilia: ambas integrantes realizaron modificaciones sobre los estilos de la cancha y del formulario de acceso, encontraron un conflicto al integrar sus trabajos y acordaron conservar sus cambios en los puntos discutidos.

Para evitar confundir una explicación pedagógica con un registro histórico, se distinguen tres niveles de información. Los hechos verificados son los que pueden observarse en archivos, commits y diferencias. El relato del equipo aporta el contexto de la decisión que Git no registra por sí mismo. Las recomendaciones describen una forma de trabajar en futuras integraciones y no se presentan como comandos que necesariamente se ejecutaron durante el incidente.

1.1. Identificación de la copia revisada

La rama local activa al comenzar la revisión es `rama-emi` y apunta al commit de fusión `985ae98`. El repositorio compartido indicado por la usuaria es Trabajo_Final_Integrador_SUM, rama-emi. La consulta web no permitió recuperar su contenido.

Además, la configuración local de `origin` utiliza otra dirección: `https://github.com/emigo5290-boop/Trabajo-Final.-Tecnicatura-en-Programaci-n..git`. Esta diferencia se registra como una limitación de identificación: no se comprobó si los repositorios fueron renombrados, si son copias o si existe otra relación entre ellos. Las conclusiones sobre commits y archivos de este informe corresponden a la copia local efectivamente inspeccionada.

Las referencias `origin/main`, `origin/rama-emi` y `origin/esme-rama` mencionadas más adelante son las referencias de seguimiento almacenadas localmente. No equivalen a una consulta actual del servidor. No se realizaron publicaciones, cambios de ramas ni una nueva fusión para elaborar este documento.

1.2. Una precisión necesaria sobre el conflicto

El relato describe una diferencia de ubicación del texto “Iniciar sesión”. En el historial verificable, la propiedad discutida pertenece a `.auth-card h1`, que selecciona el título del formulario. No selecciona el botón que envía los datos. La versión de Emilia contiene `text-align: center`; la otra versión contiene `text-align: start`.

En esta página en español, con dirección de escritura de izquierda a derecha, `start` corresponde al inicio de la línea, es decir, a la izquierda. No se encontró `text-align: right` en ese cambio concreto. La referencia a la derecha se conserva como parte del recuerdo del equipo, pero la explicación del código utiliza los valores registrados en Git.

Esta distinción permite describir correctamente dos decisiones independientes: el título centrado que se conservó al fusionar y la distribución de los botones del login, con inicio de sesión a la izquierda, registro a la derecha y recuperación de contraseña centrada debajo.

2. Contexto funcional y alcance del sistema

La aplicación representa un sistema de consulta y gestión de reservas de la cancha del SUM Municipal de Bella Vista. Reúne una vista pública y paneles específicos para cliente, administración y prensa. Esta diversidad de funciones explica por qué el proyecto necesita una organización que permita localizar cada responsabilidad sin concentrar todo en un único archivo.

La vista pública muestra el estado de la cancha, un calendario de disponibilidad, horarios, actividades e información institucional. El cliente puede registrarse, verificar su cuenta dentro de la demostración, solicitar una reserva, revisar una nota, descargar un PDF y consultar sus solicitudes. Administración revisa solicitudes, decide su aprobación o rechazo, consulta reservas, genera informes y administra el estado de la cancha. Prensa publica actividades y actualiza la información pública.

Se trata de una aplicación de frontend construida con Vue. El almacenamiento se realiza en el navegador y no existe, dentro de este repositorio, un backend que centralice datos de múltiples dispositivos. Los registros persistidos en `localStorage` pertenecen al origen del navegador que ejecuta la aplicación.

Este alcance permite demostrar los recorridos funcionales y las reglas de reserva sin desplegar un servidor de negocio. A cambio, la sesión, los permisos y las validaciones del frontend no constituyen una autenticación de producción. La documentación del proyecto reconoce expresamente esta limitación. En un sistema real, la autorización y la consistencia de las reservas deben controlarse también en el servidor.

La organización elegida debe evaluarse teniendo en cuenta ese alcance. El objetivo actual es mantener una aplicación demostrable, comprensible y modificable, con responsabilidades separadas. No se justifica describirla como una arquitectura de microservicios ni como un sistema distribuido porque esos elementos no están implementados.

3. Estructura del repositorio y criterios de organización

El repositorio utiliza una estructura por responsabilidades técnicas. Las pantallas se agrupan en `views`, los elementos visuales reutilizables en `components`, los casos de uso en `application`, las reglas del negocio en `domain` y las integraciones con recursos externos al dominio en `infrastructure`.

La siguiente estructura resume los elementos relevantes. Es una representación de las carpetas de trabajo, no una lista exhaustiva de todos los archivos presentes:

```
raíz del proyecto/
  README.md
  package.json
  package-lock.json
  vite.config.js
  index.html
  .gitignore
  public/
    fonts/
    img/
    theme.js
  src/
    main.js
    App.vue
    config.js
    application/
    domain/
    infrastructure/
    components/
    views/
    router/
    styles/
  tests/
  docs/
```

3.1. Archivos de la raíz

`README.md` cumple la función de puerta de entrada: explica cómo ejecutar el proyecto, qué funciones ofrece y cuáles son sus límites. Este archivo permite que una integrante nueva pueda comenzar sin necesitar una explicación oral de cada paso.

`package.json` declara dependencias, comandos y requisitos de ejecución. Su acompañante, `package-lock.json`, registra la resolución concreta de las dependencias. La presencia de ambos facilita que las dos integrantes instalen una base consistente mediante `npm ci`.

`vite.config.js` concentra la configuración de desarrollo y compilación. `index.html` contiene el elemento donde se monta Vue, carga el script inicial del tema y enlaza la hoja principal de estilos. Estos archivos configuran la aplicación; no deberían acumular reglas de reservas ni contenido de pantallas completas.

`.gitignore` excluye `node_modules`, `dist`, `.DS_Store` y `.env.local`. Las dependencias instaladas y los archivos compilados se pueden regenerar. Mantenerlos fuera de Git reduce diferencias sin valor para la revisión y evita mezclar el código fuente con los productos de su ejecución.

Se observó un archivo `desktop.ini` versionado. No participa en la aplicación Vue: es un archivo de personalización del entorno Windows. Como mejora de mantenimiento, se recomienda que el equipo acuerde retirarlo del versionado y agregarlo a las exclusiones. Esa limpieza no se realizó durante la elaboración de este informe.

3.2. Recursos públicos

`public/fonts` contiene las fuentes del diseño y `public/img` contiene el logo. Mantener estos recursos identificados evita depender de archivos guardados solamente en la computadora de una integrante. También permite reconocer qué archivos forman parte de la identidad visual del sistema.

`public/theme.js` tiene una responsabilidad diferente de la lógica de negocio: aplica la preferencia visual antes de que Vue termine de cargar. Esta ubicación y su carga temprana ayudan a que el primer renderizado use el tema adecuado. La selección posterior se expone mediante `ThemeSelector.vue`.

3.3. Código, pruebas y documentación

`src` contiene el código que implementa la aplicación. `tests` contiene verificaciones automatizadas de servicios, reglas, almacenamiento y tema. `docs` conserva explicaciones sobre arquitectura y validación, además de este informe.

Separar documentación y pruebas del código de las pantallas favorece la lectura. La explicación de una decisión no necesita convertirse en un comentario extenso dentro de un componente, y los datos de una prueba no se mezclan con la información que ve el usuario.

La estructura no elimina por sí misma los conflictos. Su valor es reducir la cantidad de lugares que deben tocarse para realizar una tarea y facilitar la identificación del alcance de un cambio. Una modificación de alineación debería concentrarse en los estilos correspondientes; una nueva regla de disponibilidad debería localizarse en las reglas y servicios de reservas.

4. Responsabilidades de las capas y recorrido de una reserva

4.1. Dominio: reglas independientes de la pantalla

`src/domain` reúne conceptos y operaciones del problema que se está resolviendo. `date.js` proporciona operaciones de fechas y horas; `availability.js` contiene reglas de disponibilidad y estados de la cancha; `validation.js` organiza validaciones; `note.js` construye los datos de la nota formal.

Esta separación permite comprobar una regla sin abrir el navegador. Por ejemplo, la superposición entre dos intervalos horarios puede verificarse a partir de datos de entrada y un resultado esperado, sin renderizar un calendario.

También evita que dos pantallas implementen versiones distintas de la misma regla. La consulta de disponibilidad y la aprobación administrativa necesitan criterios compatibles: no debería considerarse libre un horario en una pantalla y ocupado en otra por haber duplicado la lógica.

4.2. Aplicación: coordinación de los casos de uso

`src/application/auth.js` organiza registro, ingreso, verificación y recuperación. `reservations.js` coordina la validación, creación y decisión sobre solicitudes. `content.js` reúne operaciones sobre información pública, actividades, estado de la cancha y acceso de prensa.

Estos servicios combinan reglas con acceso a datos y permisos. Por ejemplo, solicitar una reserva exige un cliente verificado, una actividad permitida, observaciones dentro del límite y un horario disponible. Aprobarla exige el rol administrativo y una nueva comprobación del horario.

La revalidación al aprobar es relevante: entre la solicitud y su aprobación puede haber cambiado el estado de la cancha o haberse aprobado otra reserva. La vista no debe asumir que una validación realizada antes sigue siendo suficiente para una decisión posterior.

`context.js` conecta esos servicios con el estado reactivo de Vue. Expone sesión, usuario, reservas, actividades, información y avisos. Por esta razón, no corresponde afirmar que toda la carpeta `application` es independiente de Vue: los servicios de casos de uso no lo importan, pero el contexto reactivo sí.

4.3. Infraestructura: mecanismos concretos

`src/infrastructure/storage.js` implementa la lectura y escritura en el almacenamiento del navegador. `password.js` concentra el tratamiento de contraseñas de la demostración. `pdf.js` encapsula el uso de jsPDF. `flyer.js` procesa imágenes y `seed.js` crea reservas de ejemplo cuando corresponde.

La ventaja de esta ubicación es que las pantallas no necesitan conocer el formato de cada clave de `localStorage` ni todos los detalles de generación de un PDF. Invocan una operación del contexto o de un servicio, mientras el mecanismo concreto queda concentrado en el adaptador correspondiente.

El almacenamiento ofrece el contrato `read(key, fallback)` y `write(key, value)`. Incluye respaldo en memoria si el almacenamiento del navegador no está disponible. La aplicación puede seguir funcionando durante esa ejecución y avisar que no podrá conservar los datos de manera durable.

El contrato actual es síncrono. Una migración a un servidor HTTP requerirá revisar los servicios para trabajar con promesas y estados de carga y error. La separación facilita identificar dónde hacer ese trabajo, pero no convierte una integración de red en un reemplazo automático de una sola línea.

4.4. Vistas y componentes

Las vistas representan recorridos o pantallas: `PublicView.vue`, `AuthView.vue`, `BookingView.vue`, `ReservationsView.vue`, `ActivitiesView.vue` y las demás vistas de cada función. Una vista organiza contenido y enlaza eventos con operaciones de la aplicación.

Los componentes representan piezas con una responsabilidad visual o interactiva más acotada. `AvailabilityCalendar.vue` presenta calendario y horarios; `FormField.vue` unifica campos y mensajes; `AppModal.vue` gestiona diálogos; `CourtStatus.vue` muestra el estado de la cancha; `FormalNote.vue` presenta la nota; `AppBrand.vue` concentra logo y nombre.

Reutilizar estos componentes reduce diferencias accidentales. Cuando se corrige el foco de la marca o el ancho de un campo compartido, el ajuste alcanza las pantallas que utilizan esa pieza. Esa misma propagación obliga a revisar su impacto: una regla global puede mejorar o perjudicar muchas vistas a la vez.

`AuthView.vue` reúne distintos modos de acceso en un mismo componente. Los campos y títulos cambian según la ruta. Esto evita duplicar formularios parecidos, aunque exige que los estilos específicos del login se limiten a sus acciones y no alteren innecesariamente registro o recuperación.

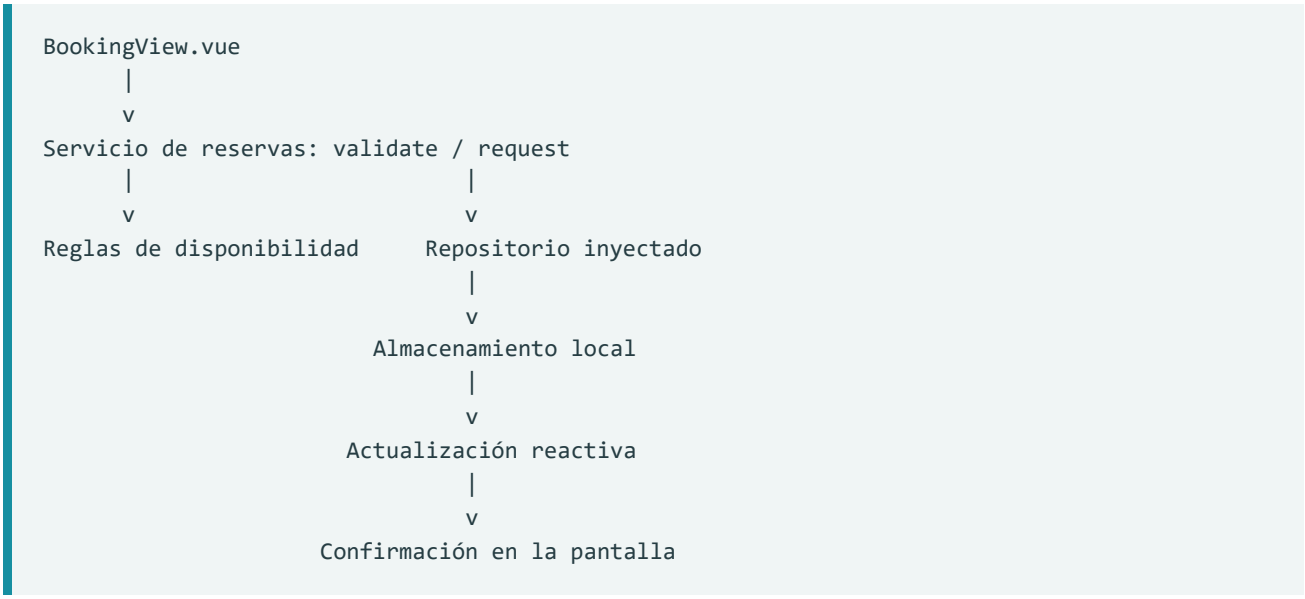
4.5. Composición y navegación

`main.js` crea los adaptadores, construye la aplicación, crea el router y proporciona las dependencias a Vue mediante `provide`. Los componentes recuperan el contexto con `useApp`, que utiliza `inject`. Este punto de composición permite ver en un solo lugar qué implementaciones concretas utiliza el programa.

`router/index.js` declara rutas, menús y guardas por perfil. Una persona sin la sesión correspondiente es dirigida al acceso o al panel asociado a su rol. Las guardas ordenan la navegación del frontend; no sustituyen los controles de seguridad de un backend.

La aplicación utiliza navegación con hash. Una ruta conceptual como `/login` aparece en el navegador como `/#/login`. Esta elección permite servir el sitio estático sin necesitar una regla de servidor específica para cada ruta interna, con el costo de que la URL conserva el símbolo `#`.

4.6. Recorrido completo de una solicitud



La vista recoge fecha, horas y actividad. El servicio verifica permisos, datos y disponibilidad. Si la solicitud es válida, genera sus identificadores y la guarda como pendiente. La escritura provoca una actualización del contexto, y Vue vuelve a representar la información necesaria.

La nota utiliza datos organizados por `domain/note.js`. La presentación en pantalla y la exportación a PDF no necesitan reconstruir de manera independiente todo el texto de la solicitud. Esta coordinación ayuda a mantener consistentes los documentos que consulta el cliente.

5. Decisiones de tecnología, dependencias y mantenimiento

El proyecto puede describirse como un monolito modular de frontend: una aplicación con un punto de entrada y un despliegue, organizada internamente en módulos. Que Vite produzca varios archivos JavaScript no significa que existan varios sistemas independientes.

La siguiente tabla resume decisiones observables y la justificación técnica que aportan. Cuando no existe un acta de decisión del equipo, la justificación se presenta como evaluación de la solución implementada, sin atribuir conversaciones que no quedaron registradas.

Decisión	Beneficio para este proyecto	Costo o límite
Vue 3 con componentes `.vue`	Permite actualizar la interfaz desde el estado y reutilizar piezas visuales.	Exige comprender reactividad, propiedades y eventos.
Una aplicación con Vue Router	Reúne los recorridos en una navegación consistente.	Las guardas del cliente no protegen datos de servidor.
Organización por capas	Hace reconocibles las reglas, los casos de uso y los mecanismos técnicos.	Requiere mantener disciplina en las dependencias.
Servicios con dependencias inyectadas	Facilita usar almacenamiento y relojes de prueba.	Los contratos deben documentarse y respetarse.
Contexto reactivo central	Permite compartir sesión y datos sin repetir acceso al almacenamiento en cada vista.	Un contexto demasiado grande puede convertirse en un punto de acoplamiento.
Persistencia local de demostración	Permite ejecutar y presentar el sistema sin servidor de negocio.	No comparte datos entre dispositivos ni resuelve concurrencia de producción.
jsPDF encapsulado	Separa la generación documental de los formularios.	Deben verificarse formato, saltos de página y contenido.
Estilos comunes y por área	Favorece una identidad visual consistente.	Los selectores globales necesitan revisión transversal.
Pruebas con `node:test`	Permite verificar reglas y servicios con las herramientas del proyecto.	No sustituye la inspección visual de la interfaz.

5.1. Relación con principios de diseño

La responsabilidad única se observa cuando el cálculo de disponibilidad, el formulario y la generación de PDF pertenecen a módulos diferentes. La inversión de dependencias aparece cuando un servicio recibe un repositorio y un reloj en lugar de crear directamente todos sus mecanismos.

La posibilidad de sustituir implementaciones existe mientras se respete su contrato. No es necesario introducir jerarquías de clases para explicar ese beneficio. En este proyecto, las funciones de creación de servicios y los objetos recibidos como parámetros son suficientes para separar varias dependencias.

Estas decisiones no implican que la arquitectura sea perfecta o que aplique todos los principios de manera absoluta. El contexto reactivo conoce Vue, el almacenamiento sigue siendo local y los servicios usan un contrato síncrono. Una explicación académica sólida describe tanto las ventajas como los límites reales de la implementación.

5.2. Dependencias y ejecución reproducible

La aplicación declara Vue, Vue Router y jsPDF como dependencias de ejecución. Vite, su integración con Vue y Prettier aparecen como herramientas de desarrollo. El requisito de Node está expresado en `package.json` como una versión igual o superior a `22.12.0`.

Compartir el archivo de bloqueo ayuda a que dos instalaciones no resuelvan silenciosamente combinaciones distintas. Sin embargo, reproducibilidad también requiere documentar los comandos y mantener el repositorio libre de cambios locales no revisados. Un archivo de bloqueo no reemplaza una comunicación clara sobre qué rama se está usando.

6. Organización de los estilos y conservación del diseño

Los estilos se importan desde `src/styles/app.css`. La base común reside en `general.css`; los paneles comparten `panel.css`; las áreas pública, de acceso, de administración, de cliente y de prensa tienen hojas identificables.

Hoja	Responsabilidad principal
`general.css`	Paleta, tipografías, base, encabezado, botones, marcador, calendario, campos y diálogos.
`panel.css`	Estructura de los paneles, navegación, bloques, tarjetas y nota formal.
`index.css`	Información y recursos de la vista pública.
`auth.css`	Tarjetas y acciones de los recorridos de acceso.
`administracion.css`	Indicadores, barras, tablas y opciones administrativas.
`cliente.css`	Elementos específicos de disponibilidad y resumen del cliente.
`prensa.css`	Presentación de flyers y elementos de prensa.
`app.css`	Importación de hojas y reglas compartidas de integración.

6.1. Variables de color y alcance de un cambio

La paleta utiliza variables CSS. El fondo del marcador público se obtiene de `--score-bg`, que tiene un valor para el tema claro y otro para el oscuro. Esto permite cambiar la identidad del marcador desde una definición central y evita repetir el mismo color en distintos componentes.

El alcance debe leerse con precisión. La variante administrativa `.marcador--admin` utiliza `var(--panel)` como fondo. Por eso, cambiar `--score-bg` no equivale a cambiar cualquier tarjeta que muestre datos de la cancha. El color depende del selector y de la variante utilizada.

Una ventaja de centralizar variables es la consistencia; un costo es que varias integrantes pueden trabajar sobre el mismo bloque de `:root`. Precisamente por ser un punto compartido, una decisión de paleta requiere coordinación y una revisión visual de los componentes que dependen de ella.

6.2. Ajustes responsive y riesgo de reglas generales

La revisión previa detectó campos con un ancho fijo de `680px` y botones desplazados mediante `left` y `top`. Esas reglas podían producir una posición aceptable en un ancho concreto y, al mismo tiempo, desplazar elementos fuera de pantalla en otros dispositivos.

El resultado integrado utiliza `box-sizing: border-box`, campos con `width: 100%`, límites de tamaño y distribuciones de Grid y Flexbox. El calendario cambia su número de columnas según el ancho disponible en el contenido. Esto importa en paneles de escritorio, donde una barra lateral reduce el espacio aunque la ventana sea amplia.

La modificación responsive debe conservar la intención del diseño. Reemplazar una posición fija no autoriza a redistribuir arbitrariamente las acciones. En el login, el criterio acordado es mantener el botón de inicio a la izquierda, el enlace de registro a la derecha y la recuperación centrada debajo.

6.3. Título, botones y recuperación: tres reglas distintas

El título del formulario está controlado por `.auth-card h1`. La fila de botones utiliza `.login-actions`, con dos columnas que pueden reducirse sin desbordar. El enlace de recuperación utiliza `.login-recovery` y se centra en una fila independiente.

```
.auth-card h1 {
  font: 900 2.2rem/1.05 var(--display);
  margin-bottom: 18px;
  text-align: center;
}

.auth-card .login-actions {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  align-items: stretch;
}

.login-actions > .btn {
  min-width: 0;
  padding-inline: clamp(10px, 2vw, 20px);
  justify-self: start;
}

.login-actions > .btn-sec {
  justify-self: end;
}

.auth-card > .login-recovery {
  justify-content: center;
}
```

Este fragmento corresponde al estado integrado analizado. Permite explicar por qué centrar el título no centra automáticamente el botón: actúan selectores diferentes sobre elementos diferentes. También muestra que la resolución visual y la adaptación responsive pueden coexistir.

7. Trabajo colaborativo y cronología de las ramas

Git registra versiones del código mediante commits. Las ramas permiten que cada integrante desarrolle una línea de trabajo antes de integrarla con la otra. Trabajar en ramas distintas reduce interferencias inmediatas, pero no impide que ambas modifiquen el mismo archivo.

En la copia revisada, la referencia local `main` permanece en el commit inicial `bca8974`. Los trabajos posteriores aparecen en las líneas asociadas a Emilia y Esmeralda. Este es un dato del historial disponible; no demuestra que exista una política formal de publicación desde `main` ni que se haya completado una integración final allí.

7.1. Cronología relevante

Las horas de la tabla se expresan con el desplazamiento `UTC-03:00`, correspondiente a los registros de los commits. Los nombres se conservan como aparecen en sus metadatos, sin incluir direcciones de correo.

Fecha y hora	Commit	Autora registrada	Cambio relevante
08/10/2026 14:54	`bca8974`	Emilia Gomez	Incorpora la maqueta Vue del sistema.
13/10/2026 12:53	`df25e90`	EsmeraldaMarquez	Agrega modo oscuro y controles de tema.
13/10/2026 14:46	`4c80b63`	EsmeraldaMarquez	Cambia fuentes de estados, títulos y párrafos.
13/10/2026 16:35	`a068054`	Emilia Gomez	Modifica colores de la cancha y posiciones de elementos.
13/10/2026 17:47	`70f280a`	Emilia Gomez	Cambia logo, botones y anchos; es la base común relevante.
13/10/2026 18:47	`c5195c8`	Emilia Gomez	Integra mejoras responsive, enlaces de marca y acciones del formulario.
13/10/2026 18:48	`3ecfb48`	EsmeraldaMarquez	Agrega alineación centrada al título de acceso.
13/10/2026 19:05:13	`c951404`	EsmeraldaMarquez	Cambia esa alineación a `start` y modifica `general.css`.

Fecha y hora	Commit	Autora registrada	Cambio relevante
13/10/2026 19:05:41	`4266549`	Emilia Gomez	Agrega alineación centrada al título en su línea de trabajo.
13/10/2026 19:28	`985ae98`	EsmeraldaMarquez	Registra la fusión de ambas líneas.

Los tiempos cercanos de los dos commits de las 19:05 ilustran el trabajo paralelo, aunque el conflicto no depende de que se hayan realizado en el mismo minuto. Lo determinante es que se modificaron versiones derivadas de una base común y que los resultados no eran compatibles en algunos bloques.

7.2. Grafo simplificado de la integración

```
                   3ecfb48 ---- c951404
                  /                  \
70f280a ----------+                   985ae98
                  \                  /
                   c5195c8 ---- 4266549

Línea superior: commits de Esmeralda.
Línea inferior: commits de Emilia.
Commit de integración: 985ae98.
```

El commit de fusión tiene dos padres: primero `c951404`, segundo `4266549`. Su base común es `70f280a`. Esas relaciones permiten reconstruir qué versión aportó cada cambio aun cuando los nombres de las ramas se muevan con commits posteriores.

La referencia local `rama-emi` ya apunta al resultado integrado. La referencia de seguimiento `origin/rama-emi` todavía apunta a `4266549` y `origin/esme-rama` a `985ae98`, dentro de esta copia. Una etiqueta de rama no debe confundirse con un identificador inmutable de la versión; para respaldar el informe se utilizan los hashes de los commits.

8. Descripción del conflicto: relato y evidencia técnica

8.1. El problema contado por el equipo

Según Emilia, ambas integrantes cambiaron aspectos de las mismas zonas de la interfaz: el color de la cancha y la alineación del texto de acceso. Al integrar sus trabajos surgió un conflicto y se acordó mantener sus decisiones visuales.

El episodio puede entenderse en dos planos. En el plano del diseño, existían variantes diferentes y era necesario acordar un resultado. En el plano del control de versiones, Git encontró fragmentos que no podía combinar automáticamente. Los dos planos se relacionan, pero no todo desacuerdo visual produce por sí mismo un conflicto textual.

8.2. Comparación de versiones verificadas

Aspecto	Base `70f280a`	Línea Esmeralda `c951404`	Línea Emilia `4266549`	Fusión `985ae98`
Fondo claro del marcador	`#c8e6ca`	`#c8e6ca`	`#c9f3cc`	`#c9f3cc`
Alineación explícita de `.auth-card h1`	Sin declaración	`start`	`center`	`center`
Margen inferior del título	`6px`	`6px`	`18px`	`18px`
Ancho común de campos	`680px`	`680px`	`100%`	`100%`
Altura mínima común de campos	`40px`	`40px`	`48px`	`48px`
Bloque final de estilos comunes	Presente	Eliminado	Presente y ajustado	Presente y ajustado

La tabla confirma el resultado visual elegido. También muestra un matiz: respecto de la base común, el valor `--score-bg` solamente cambia en la línea de Emilia. La otra línea conserva el valor de la base en ese token específico.

Por lo tanto, el historial respalda que se conservó el color de Emilia, pero no respalda describir ese token como dos cambios incompatibles sobre la misma línea durante esta fusión. Puede haber existido una discusión de diseño o ediciones intermedias que no quedaron registradas; el informe no las convierte en evidencia de Git.

8.3. Por qué editar el mismo archivo no basta para explicar el conflicto

Git compara cambios respecto de una base común. Cuando los cambios son compatibles puede integrarlos; cuando afectan de forma incompatible la misma zona necesita una resolución. Los marcadores muestran alternativas y no expresan cuál es la mejor decisión visual. Esta distinción se corresponde con la explicación de integración y conflictos de la documentación oficial de Git.

En este caso, la causa comprobable de la discusión del título es directa: una versión incorpora `start` y la otra `center` en el bloque del mismo encabezado. Ninguna herramienta de control de versiones puede deducir, a partir del CSS, cuál de esas dos preferencias acordó el equipo.

En `general.css` el problema fue más amplio. Una línea conservaba el ancho fijo y eliminaba el bloque final del archivo; la otra cambiaba los tamaños de los campos y conservaba reglas posteriores para errores, avisos, diálogos y otros elementos. La integración requería revisar el contenido completo afectado, no solamente elegir un color.

9. Reconstrucción de los bloques conflictivos

9.1. Método de reconstrucción

Se consultaron los dos padres de `985ae98` y se obtuvo su base común mediante `git merge-base`. Después se utilizó la forma de tres argumentos de `git merge-tree`, que muestra una integración de esos árboles sin reemplazar el contenido de trabajo. Esta modalidad de inspección está descrita en la referencia de git merge-tree.

```
git merge-base c951404 4266549
git merge-tree 70f280a c951404 4266549
```

La salida permite reconstruir bloques conflictivos a partir de los commits disponibles. No es una captura de la pantalla que vieron las integrantes ni un registro de los botones que pulsaron en su editor. El contenido y el resultado sí se pueden contrastar con los archivos del commit final.

9.2. Conflicto en `auth.css`

El fragmento siguiente reproduce el contenido del bloque reconstruido, sin los prefijos de líneas del diff. En esta comparación, `.our` corresponde al primer argumento de rama, `c951404`, y `.their` a `4266549`.

```
.auth-card h1 {
  font: 900 2.2rem/1.05 var(--display);
<<<<<<< .our
  margin-bottom: 6px;
  text-align: start;
=======
  margin-bottom: 18px;
  text-align: center;
>>>>>>> .their
}
```

El conflicto abarca un bloque, no necesariamente una única línea aislada. En él se combinan un cambio de espaciado y otro de alineación. La versión de Emilia aporta tanto el centrado como el margen de `18px`, y ambos aparecen en la versión integrada.

Los marcadores `<<<<<<<`, `=======` y `>>>>>>>` son indicadores de comparación. No forman parte de un estilo válido que deba permanecer en la aplicación. Resolver el bloque exige sustituir las alternativas por una regla final coherente.

9.3. Conflicto en `general.css`

En este archivo se reconstruyó una incompatibilidad al final de la regla común de campos y en los estilos que la siguen. El siguiente extracto es abreviado: indica la continuación mediante un comentario explicativo y no pretende transcribir todas las líneas de la salida.

```
<<<<<<< .our
  width: 680px;
  min-height: 40px;
}
=======
  width: 100%;
  min-width: 0;
  min-height: 48px;
}
textarea {
  min-height: 96px;
  resize: vertical;
}
/* Continúan las reglas de campos, errores, diálogos y avisos. */
>>>>>>> .their
```

La diferencia de tamaño se relaciona con el trabajo responsive. El ancho fijo podía exceder una pantalla pequeña; `width: 100%` permite que el campo siga el ancho de su contenedor. `min-width: 0` evita que el tamaño mínimo del elemento fuerce ciertos desbordes dentro de una distribución flexible.

El bloque eliminado en una de las líneas incluía reglas utilizadas por otras pantallas. No es posible deducir del diff si su eliminación fue intencional o accidental. Lo comprobable es que la fusión conserva esas reglas desde la línea de Emilia y mantiene sus ajustes.

Este hallazgo amplía la explicación del incidente: además de una preferencia de alineación, había que conservar una estructura de estilos compartida por formularios y diálogos. Una resolución que revisara únicamente el título podría dejar incompleta la integración.

9.4. El color de la cancha dentro de la integración

El valor final del tema claro es:

```
:root {
  --score-bg: #c9f3cc;
}
```

En el tema oscuro se conserva otro valor, `#0b171c`. Esto confirma que la elección documentada corresponde al fondo claro del marcador y no a la eliminación del modo oscuro.

El cambio de `#c8e6ca` a `#c9f3cc` aparece en la diferencia de la línea de Emilia respecto de la base. En la reconstrucción no queda encerrado por marcadores de conflicto. Es, por tanto, un cambio integrado y una decisión visual verificable, pero no un bloque textual conflictivo demostrado en esta fusión.

10. Resolución adoptada y estado final

10.1. Acuerdo del equipo

El acuerdo relatado fue conservar las decisiones de Emilia en los aspectos discutidos. El mensaje del commit `985ae98` expresa esa elección: “fusiona la rama de emilia porque es la adecuada con los cambios correctos”. El mensaje documenta la intención de la integración; la comparación de archivos permite comprobar qué quedó efectivamente guardado.

Conservar una propuesta es una decisión de equipo. El centrado de un título no es universalmente superior a otra alineación: resulta adecuado cuando responde al diseño acordado. En cambio, mantener los campos dentro de su contenedor sí responde a un problema observable de adaptación a distintos anchos.

10.2. Resultado concreto

La versión integrada contiene el título centrado y su margen inferior de `18px`. Conserva el fondo claro `#c9f3cc` en el marcador, los campos de ancho flexible y las reglas comunes posteriores. También contiene las mejoras de navegación de la marca y la distribución responsive de las acciones del login.

La fila de acciones del login mantiene el inicio de sesión a la izquierda y el registro a la derecha. La recuperación de contraseña aparece centrada debajo. Este punto debe explicarse separado del conflicto del título, porque elegir `text-align: center` en un `h1` no decide la ubicación de los botones.

10.3. No se descartó todo el aporte de una integrante

La comparación entre el resultado `985ae98` y su segundo padre `4266549` muestra diferencias solamente en `src/styles/general.css`: tres líneas agregadas y dos eliminadas. Allí se conserva una modificación de `--ink`, se agrega la variable `--ink-` y se corrige un comentario sobre la tipografía.

Esto demuestra que la fusión no es una copia absolutamente idéntica de toda la rama de Emilia. En los puntos discutidos se conservaron sus cambios, mientras otros cambios de la línea de Esmeralda también permanecieron. La frase “nos quedamos con mis cambios” debe entenderse referida al acuerdo sobre los aspectos en conflicto.

El propio historial compartido ya incluía aportes de ambas antes de la divergencia, como el modo oscuro y las fuentes. Resolver un conflicto a favor de una alternativa no elimina esos antecedentes ni la autoría del trabajo previo.

10.4. Aspectos que Git no permite reconstruir

El historial no registra si se usó Visual Studio Code, otro editor, una interfaz gráfica de Git o edición manual para resolver los bloques. Tampoco prueba el orden exacto de clics, los intentos descartados ni la conversación que llevó al acuerdo.

Sí registra las versiones de entrada, el resultado y la relación entre ambos historiales. Por ello, este informe explica una resolución verificable por contenido y ofrece un procedimiento equivalente como referencia, sin inventar una secuencia operativa histórica.

11. Procedimiento de resolución y verificación

11.1. Procedimiento explicativo

Un procedimiento adecuado comienza por identificar qué se intenta integrar y comprobar el estado del trabajo local. Luego se revisan los archivos marcados, se compara el propósito de cada variante y se redacta una versión final que respete el acuerdo. Finalmente se comprueban las diferencias y se registra la integración. La documentación de git merge describe el estado de conflicto y su continuación después de resolver los archivos.

Para este caso, la revisión de contenido debía atender al menos cuatro puntos: título del acceso, fondo del marcador, dimensiones de campos y conservación del bloque final de estilos. Elegir automáticamente un archivo completo sin leerlo podía descartar modificaciones compatibles de la otra integrante.

Las expresiones “actual” y “entrante”, o `ours` y `theirs`, dependen de la dirección de la operación. No significan de forma permanente “Emilia” y “Esmeralda”. En la reconstrucción realizada aquí, el primer lado corresponde a `c951404` y el segundo a `4266549`; al invertir la comparación también se invierten esas etiquetas.

11.2. Ejemplo equivalente de trabajo

Los siguientes comandos son una guía ilustrativa para una integración futura con ramas preparadas. No se ejecutaron para redactar el informe y no se deben repetir sobre el estado actual como si el conflicto siguiera pendiente.

```
git status
git branch --show-current
git log --graph --oneline --decorate --all
```

Después de iniciar la integración en la rama elegida por el equipo, se revisan los archivos afectados. Para el caso estudiado, las reglas finales deben contener `text-align: center`, el color seleccionado y los estilos responsive conservados.

```
git diff
git diff --check
git add src/styles/auth.css src/styles/general.css
git diff --cached
```

El paso de preparación registra la resolución de los archivos, pero no demuestra que la interfaz sea correcta. Antes de cerrar la integración corresponde compilar, ejecutar las pruebas relevantes y revisar las pantallas afectadas.

```
npm test
npm run build
git commit
```

En Windows, si PowerShell impide ejecutar el script `npm.ps1`, puede utilizarse `npm.cmd` para invocar el mismo gestor. Esa particularidad del entorno no cambia el significado de las comprobaciones del proyecto.

11.3. Comprobaciones realizadas para este informe

Se volvieron a ejecutar las pruebas y la compilación sobre el código de la versión fusionada analizada. La compilación de producción finalizó correctamente. Las pruebas ejecutaron 18 casos: 17 aprobaron y uno falló.

El fallo corresponde al contraste del tema claro entre `reservada` y `reservada-soft`: la prueba informa una relación de aproximadamente `2,07:1` frente a `4,5:1` requerido por ese caso. La documentación anterior del proyecto ya había registrado este fallo antes de la fusión analizada.

Por tanto, el resultado de la integración puede describirse como compilable y con los conflictos textuales resueltos, pero no como una versión con toda la suite aprobada. El contraste pendiente es un hallazgo de calidad visual separado de la decisión sobre la alineación del título.

También se revisó que el código fuente no contuviera marcadores de conflicto pendientes y se comparó el resultado con ambos padres. La reconstrucción se realizó mediante inspección de árboles, sin introducir un conflicto real en la carpeta de trabajo.

11.4. Evidencia visual anterior y alcance

`docs/VERIFICACION.md` registra una revisión previa de 19 rutas en cinco anchos y ambos temas, junto con comprobaciones de navegación y diálogos. En esta misma sesión de trabajo también se había verificado específicamente la distribución del login en anchos de 320, 390, 560, 768 y 1440 px.

Esos antecedentes explican por qué se conservan las reglas responsive. No se presentan como una nueva revisión visual integral del commit `985ae98`: después de la fusión existen cambios adicionales de color y alineación, y el presente trabajo está centrado en documentar e inspeccionar esa integración.

Antes de una entrega definitiva, conviene comprobar visualmente el título centrado, los botones, ambos temas y el fondo seleccionado en el commit de entrega. La compilación confirma que el código se procesa; una prueba funcional verifica un comportamiento; la revisión visual comprueba composición y legibilidad. Las tres aportan evidencia diferente.

12. Aprendizajes y acuerdos de trabajo recomendados

12.1. Coordinar zonas compartidas

Cuando una integrante trabaja en la paleta global y otra en el mismo formulario, es útil comunicar el archivo y el selector que se está modificando. El objetivo no es prohibir cambios simultáneos, sino reconocer qué decisiones necesitan un acuerdo antes de integrarse.

En este proyecto, `general.css` es un punto especialmente compartido. Contiene variables y componentes utilizados por muchas pantallas. Una revisión de ese archivo debe contemplar el efecto global del cambio, incluso si la tarea original se presentó como una corrección pequeña.

12.2. Usar nombres precisos para describir la interfaz

Expresiones como “mover iniciar sesión” pueden referirse al título, al botón o incluso al enlace del encabezado público. Para evitar ambigüedades, el equipo puede describir la tarea como “centrar el título `h1` de la tarjeta de acceso” o “alinear el botón de envío al borde izquierdo”.

Esta precisión mejora los mensajes de commit y las revisiones. También facilita explicar el conflicto posteriormente: se puede relacionar una intención visual con un selector y un archivo concreto.

12.3. Mantener cambios revisables

Un commit resulta más fácil de entender cuando reúne cambios relacionados y su mensaje describe el comportamiento final. Si una modificación del título incluye además la eliminación de muchas reglas al final de otro archivo, esa diferencia debe revisarse antes de compartirla, aunque el mensaje sólo mencione la alineación.

El caso observado permite justificar la revisión del diff completo. No basta con leer el nombre del commit: un mensaje resume la intención, pero el parche muestra el alcance efectivo de la modificación.

12.4. Actualizar e integrar con frecuencia

Las ramas que divergen durante mucho tiempo acumulan más diferencias que revisar. Integrar cambios pequeños y comunicar la base de trabajo facilita detectar desacuerdos antes de que se mezclen con nuevas tareas.

La frecuencia no elimina todos los conflictos. Dos cambios incompatibles en la misma regla seguirán requiriendo una decisión. El beneficio es que la conversación se limita a menos modificaciones y resulta más sencillo conservar aportes compatibles.

12.5. Acordar criterios de aceptación

Para los cambios estudiados, un acuerdo verificable puede formularse así: el título queda centrado; el botón de inicio permanece a la izquierda; el registro queda a la derecha; la recuperación se centra debajo; el formulario no desborda; el marcador utiliza el color acordado en el tema claro; el modo oscuro conserva una presentación legible.

Estos criterios permiten evaluar el resultado compartido sin depender de que cada integrante recuerde una captura distinta. También permiten separar una preferencia visual de una condición funcional, como que los enlaces sigan llevando a sus destinos.

12.6. Registrar la decisión, no sólo cerrar el conflicto

Un mensaje de integración útil debe indicar qué se eligió y por qué. Por ejemplo: “Integra el título centrado y conserva los campos responsive; mantiene el fondo acordado del marcador”. Es una propuesta de mensaje más descriptivo, no el texto histórico del commit existente.

La documentación puede completar lo que un mensaje breve no explica. En este caso, dejar por escrito la diferencia entre título y botón y el alcance del conflicto de `general.css` evita que el episodio se recuerde solamente como una disputa por un color.

13. Evaluación de la organización y mejoras posibles

La organización actual permite localizar con bastante claridad las pantallas, los servicios y los mecanismos técnicos. El conflicto quedó concentrado en estilos; no fue necesario mezclar decisiones de colores con reglas de aprobación de reservas. Esa separación es una ventaja concreta de la estructura del repositorio.

También permite reconocer dónde existe acoplamiento. `general.css` reúne muchas responsabilidades visuales y puede convertirse en un lugar frecuente de edición compartida. Si el proyecto crece, se puede evaluar extraer familias de componentes visuales a hojas específicas, manteniendo en la base las variables y reglas verdaderamente generales.

Esa extracción debe hacerse por responsabilidad y uso, no simplemente para aumentar la cantidad de archivos. Dividir una regla entre varias hojas y compensarla con múltiples sobrescrituras puede empeorar el mantenimiento. La meta es que cada decisión de estilo tenga un lugar reconocible.

La variable `--ink-`, presente en la integración, no tiene consumidores `var(--ink-)` encontrados en el código revisado. Conviene revisar con el equipo si fue agregada deliberadamente o si quedó como residuo de una edición. Este informe la registra como un punto de limpieza posible; no la modifica ni le atribuye una causa que el historial no demuestra.

En el plano de colaboración, sería útil establecer cuál es la rama de entrega y qué condiciones debe cumplir antes de integrar cambios. La copia local muestra una fusión entre líneas de trabajo, pero no una integración posterior a `main`. Tampoco se verificaron reglas remotas de protección ni revisiones de pull requests, por lo que no se las presenta como prácticas ya implementadas.

En el plano de calidad, el contraste pendiente debe quedar visible en la planificación. Conservar la identidad de una paleta no impide ajustar determinados tonos para mejorar legibilidad, siempre que el equipo acuerde ese cambio como una tarea diferenciada.

La evolución hacia un backend requerirá otra etapa: almacenamiento compartido, autorización del servidor, operaciones atómicas sobre reservas y adaptación de los servicios a llamadas asíncronas. La estructura actual ofrece lugares identificables para esa evolución, aunque su implementación todavía no forma parte de la demostración.

14. Texto de síntesis para la presentación del trabajo

El repositorio del sistema del SUM Municipal se organizó por responsabilidades para que las pantallas, las reglas de negocio y los mecanismos de almacenamiento no quedaran mezclados. Las vistas presentan los recorridos de los usuarios, los componentes reúnen elementos reutilizables, los servicios coordinan los casos de uso y el dominio concentra las reglas de disponibilidad y validación. Esta distribución facilita comprender el proyecto, probar sus operaciones y localizar el alcance de una modificación.

Durante el trabajo colaborativo, Emilia y Esmeralda desarrollaron cambios sobre una base común. Las modificaciones incluyeron decisiones visuales sobre la cancha y el formulario de acceso. Al integrar las ramas se encontraron diferencias incompatibles en los estilos. La reconstrucción del historial permite

verificar un conflicto sobre el título del formulario y otro sobre dimensiones de campos y reglas compartidas en `general.css`.

El equipo acordó conservar las decisiones de Emilia en los aspectos discutidos. El resultado integrado mantiene el título “Iniciar sesión” centrado, el fondo claro seleccionado para el marcador y las reglas que permiten que los campos se adapten a sus contenedores. La distribución de los botones se conserva como una decisión separada: inicio de sesión a la izquierda, registro a la derecha y recuperación centrada debajo.

El commit `985ae98` registra la fusión de los dos historiales. Su contenido demuestra que la resolución no consistió en eliminar todo el trabajo de una integrante: conserva los cambios acordados de Emilia y también modificaciones compatibles de la línea de Esmeralda. La compilación es correcta; permanece una prueba de contraste pendiente, que ya estaba identificada antes de esta revisión.

La experiencia aporta un aprendizaje concreto sobre el trabajo en equipo. Git permite detectar y registrar diferencias, pero no decide por sí mismo cuál es el diseño que se desea conservar. La resolución requiere comparar el código, acordar criterios visuales, revisar el resultado completo y documentar las decisiones con precisión.

15. Anexos: comandos, glosario y fuentes

15.1. Comandos de inspección reproducible

Los siguientes comandos permiten revisar la evidencia de este informe mientras los commits continúen disponibles. Son consultas de historial y diferencias; no cambian la rama activa ni aplican una fusión al directorio de trabajo.

```
git status --short
git remote -v
git branch -avv
git log --graph --oneline --decorate --all
git show --no-patch --format=fuller 985ae98
git merge-base c951404 4266549
git merge-tree 70f280a c951404 4266549
git show 4266549 -- src/styles/auth.css
git show c951404 -- src/styles/auth.css
git diff 70f280a c951404 -- src/styles/general.css
git diff 70f280a 4266549 -- src/styles/general.css
git diff 985ae98^1 985ae98 -- src/styles/auth.css
git diff 985ae98^2 985ae98 -- src/styles/general.css
```

El comando `git show --cc` puede ser útil, pero su salida combinada no debe interpretarse como una lista exhaustiva de todo conflicto histórico. Si la resolución de un archivo coincide íntegramente con uno de los padres, ciertas vistas combinadas pueden no mostrarlo. En este informe se complementó esa inspección con comparaciones individuales y la reconstrucción de la base común.

15.2. Identificadores completos

Referencia	Identificador
Base común	`70f280a9e699d73373c3161e27c2f3b4de56b368`
Primer padre de la fusión	`c951404b512c1bd7b43e4aa169500e4c2f77b635`
Segundo padre de la fusión	`4266549d38a1bd4ffce6035afd469972242d66f4`
Resultado integrado	`985ae98ddb53a6b12457f5228ad3ca6987d0eace`

15.3. Glosario

Término	Significado en este informe
Repositorio Git	Código e historial de versiones del proyecto.
Repositorio de datos	Contrato de lectura y escritura utilizado por los servicios; no es lo mismo que el repositorio Git.
Commit	Registro identificable de una versión y su relación con versiones anteriores.
Rama	Referencia móvil a una línea de trabajo.
Base común	Versión compartida desde la cual se comparan dos líneas de cambios.
Merge o fusión	Integración de historiales.
Conflicto textual	Incompatibilidad que requiere resolver cómo queda un fragmento de contenido.
Conflicto de diseño	Diferencia de criterio visual o funcional que requiere acuerdo, aunque Git pueda integrar el texto.
Diff	Comparación que muestra contenido agregado, modificado o eliminado.
Responsive	Capacidad de adaptar la composición al espacio disponible.
Componente	Pieza reutilizable de interfaz con una responsabilidad delimitada.
Adaptador	Implementación concreta de un mecanismo, como almacenamiento o PDF.
Token de diseño	Variable que concentra un valor visual, como un color.

15.4. Fuentes y trazabilidad

Fuentes primarias locales: árbol de archivos y commits de la copia del proyecto, con corte en `985ae98`; `README.md`; `docs/ARQUITECTURA.md`; `docs/VERIFICACION.md`; `package.json`; `src/main.js`; `src/application`; `src/domain`; `src/infrastructure`; `src/router/index.js`; `src/views/AuthView.vue`; `src/styles/auth.css`; `src/styles/general.css`; pruebas de `tests`.

Contexto aportado por la usuaria: explicación del trabajo simultáneo con su compañera y del acuerdo de conservar sus cambios en los aspectos discutidos. Se utiliza para explicar la intención del equipo, diferenciada de los valores exactos observados en el historial.

Repositorio indicado: Trabajo_Final_Integrador_SUM, rama-emi. Se identifica como dirección aportada, no como contenido remoto recuperado durante esta revisión.

Referencias técnicas consultadas: integración y conflictos en Pro Git, referencia de git merge y referencia de git merge-tree. Las descripciones del caso, sus valores y su cronología proceden del análisis del repositorio local.

Criterio de lectura: las recomendaciones futuras no representan acciones ya realizadas; los ejemplos abreviados están identificados; los resultados de validación se informan con sus límites. El propósito es que el documento pueda sostenerse con evidencia y que otra persona pueda repetir la inspección.